

claude-windows / bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f\subagents\workflows\wf_b8703ccc-78b\agent-af2a4710d63f66ec8.jsonl`
- SHA-256: `a79238dc9b1d3b167a1aee472e5a84935ecc7f98c821c8b4b33d2d756b9cfb27`
- Source modified: `2026-06-17T14:36:12+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8703ccc-78b`
- session_id: `bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f`
```

Transcript

1. user (2026-06-17T14:35:18.778Z)

Adversarially VERIFY this code-review finding against the ACTUAL code in undefined. Default to is_real=false unless you can point to the exact line that proves the bug is real and reachable. Finding: severity=major; title="watermark now passed to VideoCompiler in legacy path (was None before)"; where=tools/generate_highlights.py:189-196; why=The old code on master called VideoCompiler(local_work_dir) with watermark defaulting to None. The new code extracts watermark=st.get('watermark') from config.json, which is the full watermark dict including {enabled:true, text:...}. This enables watermarking on reels even in the legacy mode (no --detect-from), whereas master-based reels were un-watermarked. This is a functional behavior change for all existing invocations without --detect-from.. Read the relevant file(s) and the tests. Is this a REAL, reachable problem in THIS diff (not a pre-existing issue, not a hypothetical)?

2. assistant (2026-06-17T14:35:24.754Z)

I'll analyze this code-review finding by examining the actual code in the repository to verify if this is a real, reachable problem.

3. user (2026-06-17T14:35:25.111Z)

```
1      """Batch highlight-reel runner ? orchestrates "generate highlights for these videos
2      into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
3
4      Why this exists (lessons paid for the hard way):
5      - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work
dir
6      first, then COPIED into the destination (atomically: temp name -> os.replace). The
7      destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
8      Google-Drive-synced folder once lost finished reels + logs.)
9      - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
10     client (deleted files resurrect, partial writes sync). Generate locally; publish once.
11     - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
12     (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
13     duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
14     - **Resumable & concurrency-friendly.** A video whose reel already exists in the
15     destination is skipped (unless --force); launching a second request never undoes the
16     first's published reels. The Gemini path also isolates its chunk workdir per video
17     (PR for output/chunks/<video_id>) so concurrent videos don't collide.
18
19     Usage:
20     python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
21         --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
22     python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
23     """
24     from __future__ import annotations
```

```

25
26 import argparse
27 import logging
28 import os
29 import shutil
30 import sys
31 import traceback
32 from typing import Any, Dict, List, Optional, cast
33
34 from dotenv import load_dotenv
35
36 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
37
38 from backend.config import load_config
39 from backend.infrastructure.database import Database
40 from backend.pipeline.segmenters.base import segmenter_registry
41 from backend.pipeline.stitcher.compiler import VideoCompiler
42 from backend.results import Failure
43 from backend.utils.hashing import compute_video_id
44
45 logger = logging.getLogger("highlights_runner")
46
47 # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
48 _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
49 _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
50
51
52 def _atomic_publish(local_path: str, dest_path: str) -> None:
53     """Copy local_path -> dest_path so the destination never sees a partial file.
54
55     Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
56     anything else in the destination folder.
57     """
58     os.makedirs(os.path.dirname(dest_path), exist_ok=True)
59     tmp = dest_path + ".publishing.tmp"
60     shutil.copy2(local_path, tmp)
61     os.replace(tmp, dest_path)
62
63
64 def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
65                               local_work_dir: str, db_path: str, force: bool = False,
66                               skip_ai_handoff: bool = False, wasb_stream: bool = False)
-> dict:
67     """Generate one highlight reel per input video into out_dir. Returns a summary dict.
68
69     Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
70
71     skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
handoff);
72         candidate windows become the rallies. wasb_stream: have the WASB detector decode
the
73         video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
#178).
74     """
75     os.makedirs(out_dir, exist_ok=True)
76     os.makedirs(local_work_dir, exist_ok=True)
77     needs_local = segmenter in _WSL_DETECTORS
78
79     # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
80     # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
81     # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
82     # (skip_ai_handoff) needs no key, so a missing .env is harmless there.

```

```

83     load_dotenv()
84
85     cfg = load_config()
86     # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get()).
87     # Logged by the segmenter itself once logging is configured below.
88     if skip_ai_handoff:
89         cfg.indexing.skip_ai_handoff = True
90     if wasb_stream:
91         cfg.indexing.wasb.stream_video = True
92     db = Database(db_path)
93     seg_cls = segmenter_registry.get(segmenter)
94     if seg_cls is None:
95         raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
96
97     # Combined batch log (local; published at the end).
98     fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
99     if not logging.getLogger().handlers:
100         logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
101         for h in logging.getLogger().handlers:
102             h.setFormatter(fmt)
103     batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
104     batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
105     batch_fh.setFormatter(fmt)
106     logging.getLogger().addHandler(batch_fh)
107
108     summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
109     logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
len(video_paths), out_dir, segmenter, force)
110
111
112     for i, src in enumerate(video_paths, 1):
113         name = os.path.basename(src)
114         stem = os.path.splitext(name)[0]
115         reel_name = f"{stem} - Highlights.mp4"
116         dest_reel = os.path.join(out_dir, reel_name)
117         local_reel = os.path.join(local_work_dir, reel_name)
118
119         if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
120             logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
121             summary["skipped"].append(name)
122             continue
123         if not os.path.exists(src):
124             logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
125             summary["failed"].append(name)
126             continue
127
128         run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
129         run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
130         run_fh.setFormatter(fmt)
131         logging.getLogger().addHandler(run_fh)
132         staged_src: Optional[str] = None
133         try:
134             logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
135             if needs_local:
136                 staged_src = os.path.join(local_work_dir, name)
137                 if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
138                     logger.info("localizing source (%.0f MB)...", os.path.getsize(src) /
1024 / 1024)

```

```

139         shutil.copy2(src, staged_src)
140         proc_src = staged_src
141     else:
142         proc_src = src
143
144     vid = compute_video_id(proc_src)
145     db.clear_video_data(vid)          # idempotent: no cross-run accumulation
146     db.add_video(vid, proc_src, "processed", [])
147     logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
148
149     # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
150     # main.py); cast keeps the type checker honest about that contract.
151     result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
152     if isinstance(result, Failure):
153         logger.error("segmentation FAILED: %s", getattr(result, "message",
154 result))
155         summary["failed"].append(name)
156         continue
157
158     segs = db.query_play_segments(vid, {})
159     pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
160     logger.info("rallies detected: %d", len(pairs))
161     if not pairs:
162         logger.warning("no rallies -> no reel")
163         summary["no_rallies"].append(name)
164         continue
165
166     # Compile locally, then publish atomically into the destination.
167     VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
168     _atomic_publish(local_reel, dest_reel)
169     try:
170         os.remove(local_reel)
171     except OSError:
172         pass
173     logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
174     summary["done"].append(name)
175 except Exception:
176     logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
177     summary["failed"].append(name)
178 finally:
179     logging.getLogger().removeHandler(run_fh)
180     run_fh.close()
181     # publish this video's run.log (additive; never deletes anything in out_dir)
182     try:
183         _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
184     except OSError:
185         pass
186     if staged_src and os.path.exists(staged_src):
187         try:
188             os.remove(staged_src)
189         except OSError:
190             pass
191
192     logging.getLogger().removeHandler(batch_fh)
193     batch_fh.close()
194     try:
195         _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
196     except OSError:
197         pass
198     logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
199     return summary
200
201 def _resolve_videos(args) -> List[str]:

```

```

202         if args.video:
203             return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
204         if args.videos_dir:
205             return sorted(
206                 os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
207                 if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
208             )
209         raise SystemExit("provide --videos-dir or one/more --video")
210
211
212     def main() -> None:
213         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
214         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
215         ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
216         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
217         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
218         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
219                         help="local staging dir (reels compiled here, then published)")
220         ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
221         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
222         ap.add_argument("--skip-ai-handoff", action="store_true",
223                         help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
224                             "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
225         ap.add_argument("--wasb-stream", action="store_true",
226                         help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
227                             "dumping ~46k PNGs. Only affects wasb_hybrid.")
228         args = ap.parse_args()
229         generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
230                                 local_work_dir=args.local_work, db_path=args.db,
231                                 force=args.force,
232                                 skip_ai_handoff=args.skip_ai_handoff,
233                                 wasb_stream=args.wasb_stream)
234
235     if __name__ == "__main__":
236         main()

```

4. assistant (2026-06-17T14:35:28.146Z)

Looking at line 166, I see the current code:

```

```python
VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
```

```

The finding claims that the new code extracts `watermark=st.get('watermark')` from config.json and passes it to VideoCompiler. However, I don't see that in the current code. Let me check the VideoCompiler class to understand its signature and what watermark parameter it accepts.

5. user (2026-06-17T14:35:28.357Z)

backend\pipeline\stitcher\compiler.py

6. user (2026-06-17T14:35:28.811Z)

```
a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
ef60036 feat(config): R1 milestone ? flip windowing defaults to cap=6 + R4 [DO NOT MERGE w/o
product-go]
8a45334 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF)
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)
a4d8c47 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU)
0740c66 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
(#200)
e248d2d feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
7e8f88f feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU) (#199)
a450c53 feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU)
cf8973e docs(eval): R2 design ? task-faithful tolerance-match rally metric (FOR REVIEW) (#198)
bb7b9e4 docs(eval): lock R2 open decisions per owner review (2026-06-17)
38808f0 docs(eval): R2 design ? task-faithful tolerance-match rally metric (for review)
369e90c docs(eval): golden-labeled video registry (the growing golden-finalized set) (#197)
c33cf1d docs(eval): golden-labeled video registry (the growing golden-finalized set)
c483346 docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-
set discipline (#196)
ce7eb47 docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-
set discipline
bb4d6e2 feat(eval): Tier 1 (W1.5) ? hard-cap fallback enforces the window cap on continuously-
active trajectories (#195)
```

7. assistant (2026-06-17T14:35:30.346Z)

The top commit is `a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels`. This is likely the commit being reviewed. Let me check that commit and the VideoCompiler class.

8. user (2026-06-17T14:35:30.996Z)

```
duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
- **Resumable & concurrency-friendly.** A video whose reel already exists in the
  destination is skipped (unless --force); launching a second request never undoes the
  first's published reels. The Gemini path also isolates its chunk workdir per video
  (PR for output/chunks/<video_id>) so concurrent videos don't collide.
```

Usage:

```
python -m tools.generate_highlights --videos-dir "G:\...\June 12 2026" \
  --out "G:\...\June 12 2026\Highlights" --segmenter gemini [--force]
python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
"""
```

--

```
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.results import Failure
from backend.utils.hashing import compute_video_id
```

```
logger = logging.getLogger("highlights_runner")
```

Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.

```
_WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
_VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
```

--

```
VideoCompiler(
    local_work_dir,
    begin_padding=float(st.get("begin_padding", 0.5)),
    end_padding=float(st.get("end_padding", 0.5)),
```

```

        watermark=st.get("watermark"),
    ).compile_highlights(stitch_src, pairs, reel_name)
    _atomic_publish(local_reel, dest_reel)
    try:
        os.remove(local_reel)
    except OSError:
        pass

```

9. user (2026-06-17T14:35:31.067Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13    # backend/pipeline/stitcher/, so parents[2] == backend/.
14    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16    class VideoCompiler:
17        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                    watermark: Optional[Any] = None):
19            self.output_dir = output_dir
20            self.end_padding = end_padding
21            self.begin_padding = begin_padding
22            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23            # normalized to a dict (or None) so this module stays config-package-agnostic.
24            self.watermark = self._normalize_watermark(watermark)
25            os.makedirs(output_dir, exist_ok=True)
26
27        @staticmethod
28        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29            if watermark is None:
30                return None
31            if hasattr(watermark, "model_dump"):
32                return watermark.model_dump()
33            if isinstance(watermark, dict):
34                return dict(watermark)
35            return None
36
37        @staticmethod
38        def _ff_quote(path: str) -> str:
39            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41            slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44            option parser treats ':' as the option separator, so a Windows drive-letter
45            path like `C:/dir/font.ttf` otherwise truncates the operand at `C` and
46            the encode fails with "Invalid argument". This silently disabled the
47            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48            return path.replace("\\", "/").replace("'", r'\').replace(":", r"\:")
49
50        @staticmethod

```

```

51     def _parse_time(val: Union[int, float, str]) -> float:
52         """Normalizes a timestamp value to float seconds.
53         Handles: float/int (pass-through), plain numeric strings ("12.4"),
54         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55         This mirrors the parse_time logic used in gemini.py to ensure
56         the stitcher can always consume whatever format the indexer produced."""
57         if isinstance(val, (int, float)):
58             return float(val)
59         if isinstance(val, str):
60             val = val.strip()
61             if ":" in val:
62                 # Handle MM:SS, HH:MM:SS, etc.
63                 parts = val.split(":")
64                 parts.reverse()
65                 total = 0.0
66                 for i, p in enumerate(parts):
67                     try:
68                         total += float(p) * (60 ** i)
69                     except ValueError:
70                         pass
71                 return total
72             try:
73                 return float(val)
74             except ValueError:
75                 logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76                 return 0.0
77                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
78 '{val}'. Defaulting to 0.0.")
79                 return 0.0
80
81     @staticmethod
82     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
83 float]]:
84         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
85
86         A highlight reel must never replay the same footage. The same rally can land in
87         the segment list more than once ? e.g. two detector runs accumulated for one
88         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
89         or
90         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
91         rally
92         twice. Merging any range that overlaps or abuts the previous one collapses true
93         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
94         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
95         """
96         parsed = []
97         for raw_s, raw_e in segments:
98             s = VideoCompiler._parse_time(raw_s)
99             e = VideoCompiler._parse_time(raw_e)
100             if e > s:
101                 parsed.append((s, e))
102         parsed.sort()
103         merged: List[Tuple[float, float]] = []
104         for s, e in parsed:

```

10. assistant (2026-06-17T14:35:32.841Z)

Perfect! I found the issue. In the commit, the VideoCompiler call was changed to pass the watermark parameter. Let me verify the exact lines referenced in the finding (189-196) by checking the diff:

11. user (2026-06-17T14:35:33.720Z)

commit a6adcc57290bd65c3ca7c092d0568ff02cf60378

Author: Avi Dullu <avi.dullu@gmail.com>

Date: Wed Jun 17 19:59:43 2026 +0530

fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels

The batch runner (tools/generate_highlights.py) constructed VideoCompiler bare ?
`VideoCompiler(local\_work\_dir).compile\_highlights(...)` ? so reels shipped UN-watermarked
(watermark defaults to None/off) and used the ctor's begin/end padding (0.5/1.0) instead of
config.json `stitching.{begin\_padding,end\_padding}` (0.5/0.5). Every other compile call site
threads stitching config; this one didn't. Now it reads `stitching.\*` dict-compatibly and
passes watermark + paddings through (the on-by-default branding mark).

Also add `--detect-from`: run the detector on a cheaper proxy (e.g. a 1080p downscale of a
4K source) while stitching the reel from the full-res --video. Timestamps transfer 1:1 (pure
temporal-preserving downscale), so it's the bandwidth/GPU-saver split the workflow wanted ?
detect on the proxy, deliver a 4K reel. Reel is named after the --video (the thing watched).

Tests: extend test_generate_highlights.py ? watermark+padding are threaded into
VideoCompiler
(regression lock), --detect-from keys detection off the proxy but stitches from the
original,
and _resolve_detect_from validates 1:1 pairing.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

```
diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
index ae9f045..565663a 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -63,7 +63,8 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:
--
-         # Compile locally, then publish atomically into the destination.
-         VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+         # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
+         # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
+         stitch_src = src if detect_from_paths else proc_detect
+         # Honor the configured branding watermark + segment paddings (stitching.*). Read
+         # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
+         # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
+         # ctor default paddings; this threads config.json through (the on-by-default mark).
+         st = cfg.get("stitching") or {}
+         VideoCompiler(
+             local_work_dir,
+             begin_padding=float(st.get("begin_padding", 0.5)),
+             end_padding=float(st.get("end_padding", 0.5)),
+             watermark=st.get("watermark"),
+         ).compile_highlights(stitch_src, pairs, reel_name)
+         _atomic_publish(local_reel, dest_reel)
+         try:
+             os.remove(local_reel)
@@ -209,6 +237,21 @@ def _resolve_videos(args) -> List[str]:
    raise SystemExit("provide --videos-dir or one/more --video")

+def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
+    """Resolve --detect-from into a list parallel to video_paths (or None if unused).
```

12. assistant (2026-06-17T14:35:35.143Z)

Excellent! Now let me check what the master branch (pre-change) code looked like:

13. user (2026-06-17T14:35:36.081Z)

```

Google-Drive-synced folder once lost finished reels + logs.)
- **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
  client (deleted files resurrect, partial writes sync). Generate locally; publish once.
- **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
  (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
  duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
- **Resumable & concurrency-friendly.** A video whose reel already exists in the
  destination is skipped (unless --force); launching a second request never undoes the
  first's published reels. The Gemini path also isolates its chunk workdir per video
  (PR for output/chunks/<video_id>) so concurrent videos don't collide.

```

```
--
```

```
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```

from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.results import Failure
from backend.utils.hashing import compute_video_id

```

```
logger = logging.getLogger("highlights_runner")
```

```
--
```

```

    logger.warning("no rallies -> no reel")
    summary["no_rallies"].append(name)
    continue

```

```

# Compile locally, then publish atomically into the destination.
VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
_atomic_publish(local_reel, dest_reel)
try:
    os.remove(local_reel)
except OSError:
    pass

```

14. assistant (2026-06-17T14:35:37.473Z)

Perfect! Now let me verify if there's a config.json in the project and what the watermark settings look like:

15. user (2026-06-17T14:35:38.435Z)

```

./.claude/worktrees/agent-a771ff1d85e18f2ed/config.json
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_config.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_config_wasb.py
./.claude/worktrees/bhi-execpolicy/config.json
./.claude/worktrees/bhi-execpolicy/tests/test_config.py
./.claude/worktrees/bhi-execpolicy/tests/test_config_wasb.py
./.claude/worktrees/boxhill17/config.json
./.claude/worktrees/boxhill17/tests/test_config.py
./.claude/worktrees/boxhill17/tests/test_config_wasb.py
./.claude/worktrees/docs-golden/config.json
./.claude/worktrees/docs-golden/tests/test_config.py
./.claude/worktrees/docs-golden/tests/test_config_wasb.py
./.claude/worktrees/docs-rally-quality/config.json
./.claude/worktrees/docs-rally-quality/tests/test_config.py
./.claude/worktrees/docs-rally-quality/tests/test_config_wasb.py
./.claude/worktrees/docs-today/config.json
./.claude/worktrees/docs-today/tests/test_config.py
./.claude/worktrees/docs-today/tests/test_config_wasb.py
./.claude/worktrees/hardchop/config.json
./.claude/worktrees/hardchop/tests/test_config.py

```

16. assistant (2026-06-17T14:35:39.975Z)

Let me check the actual config loading and watermark structure:

17. user (2026-06-17T14:35:40.350Z)

(Bash completed with no output)

18. user (2026-06-17T14:35:41.837Z)

No files found

19. user (2026-06-17T14:35:44.304Z)

```
./claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/loader.py
./claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py
./claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/__init__.py
./claude/worktrees/bhi-execpolicy/backend/config/loader.py
./claude/worktrees/bhi-execpolicy/backend/config/models.py
./claude/worktrees/bhi-execpolicy/backend/config/__init__.py
./claude/worktrees/boxhill17/backend/config/loader.py
./claude/worktrees/boxhill17/backend/config/models.py
./claude/worktrees/boxhill17/backend/config/__init__.py
./claude/worktrees/docs-golden/backend/config/loader.py
./claude/worktrees/docs-golden/backend/config/models.py
./claude/worktrees/docs-golden/backend/config/__init__.py
./claude/worktrees/docs-rally-quality/backend/config/loader.py
./claude/worktrees/docs-rally-quality/backend/config/models.py
./claude/worktrees/docs-rally-quality/backend/config/__init__.py
./claude/worktrees/docs-today/backend/config/loader.py
./claude/worktrees/docs-today/backend/config/models.py
./claude/worktrees/docs-today/backend/config/__init__.py
./claude/worktrees/hardchop/backend/config/loader.py
./claude/worktrees/hardchop/backend/config/models.py
./claude/worktrees/hardchop/backend/config/__init__.py
./claude/worktrees/local-noai/backend/config/loader.py
./claude/worktrees/local-noai/backend/config/models.py
./claude/worktrees/local-noai/backend/config/__init__.py
./claude/worktrees/pr-stitch/backend/config/loader.py
./claude/worktrees/pr-stitch/backend/config/models.py
./claude/worktrees/pr-stitch/backend/config/__init__.py
./claude/worktrees/r2-impl/backend/config/loader.py
./claude/worktrees/r2-impl/backend/config/models.py
./claude/worktrees/r2-impl/backend/config/__init__.py
./claude/worktrees/r4-endrule/backend/config/loader.py
./claude/worktrees/r4-endrule/backend/config/models.py
./claude/worktrees/r4-endrule/backend/config/__init__.py
./claude/worktrees/tier0-eval/backend/config/loader.py
./claude/worktrees/tier0-eval/backend/config/models.py
./claude/worktrees/tier0-eval/backend/config/__init__.py
./claude/worktrees/tier1-windowing/backend/config/loader.py
./claude/worktrees/tier1-windowing/backend/config/models.py
./claude/worktrees/tier1-windowing/backend/config/__init__.py
./claude/worktrees/w2-netcross/backend/config/loader.py
./claude/worktrees/w2-netcross/backend/config/models.py
./claude/worktrees/w2-netcross/backend/config/__init__.py
./claude/worktrees/w2-safeguard/backend/config/loader.py
./claude/worktrees/w2-safeguard/backend/config/models.py
./claude/worktrees/w2-safeguard/backend/config/__init__.py
./backend/config/loader.py
./backend/config/models.py
./backend/config/__init__.py
```

20. user (2026-06-17T14:35:45.843Z)

```

1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"

```

```

65     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66     sport: str = "badminton"
67     wsl_stage_dir: str = "~/clips"
68     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69     velocity_threshold: float = 0.02
70     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73     keep_frames: bool = False
74     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75     min_free_gb: float = 0.0
76     # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
77     # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
78     # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
79     # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
80     # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81     stream_video: bool = False
82
83
84     class FusionConfig(BaseModel):
85         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87         Every default reproduces control behaviour: the fusion segmenter persists the
88         shuttle net-crossing count (Gate A signal) and ? only when `cue_flags['person']`
is
89         true ? the person-cue features, but it does NOT gate the served handoff unless a
90         threshold is raised (`min_crossings`/`min_players` default 0 = OFF). The court
mask
91         is optional: `court_polygon=None` ? Gate B counts every detection (player-count-
only);
92         supplied ? off-court persons (spectators / adjacent courts) are excluded.
93         ""
94         # Which trajectory substrate seeds the windows (shuttle stays primary).
95         substrate: Literal["wasb", "tracknet"] = "wasb"
96         # Windowing velocity threshold for the fusion family (the engine reads
97         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99         velocity_threshold: float = 0.02
100        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101        # enabled ? so the default path never imports torch / touches the GPU.
102        cue_flags: dict[str, bool] = Field(default_factory=dict)
103        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105        min_crossings: int = Field(default=0, ge=0)
106        # Served Gate B: when the person cue is on, drop windows with median in-court
players
107        # < this. 0 = OFF.
108        min_players: int = Field(default=0, ge=0)
109        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110        court_polygon: Optional[list[list[float]]] = None
111        # Net line for the person two-sided-motion split (person features only ? the shuttle
112        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113        net_axis: Literal["x", "y"] = "y"
114        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
115

```

```

116
117     class IndexingConfig(BaseModel):
118         default_segmenter: str = "yolo_hybrid"
119         use_gpu: bool = True
120         motion_threshold: float = 0.08
121         min_segment_duration: float = 3.0
122         dead_time_merge_gap: float = 2.0
123         chunk_duration: float = 90.0
124         chunk_overlap: float = 10.0
125         detector_model: str = "fasterrcnn_resnet50_fpn"
126         detector_score_threshold: float = 0.5
127         yolo_velocity_threshold: float = 0.15
128         yolo_frame_skip: int = 3
129         yolo_tracker: str = "bytetrack.yaml"
130         yolo_prefetch_workers: int = 2
131         min_action_window_duration: float = 2.0
132         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF
133         (default ? # behaviour unchanged). When > 0, a candidate window longer than this many seconds
134         is # recursively split at its largest internal inactivity gap (?
135         ``window_min_split_gap`` s ? # the most likely rally boundary), so one window can't swallow several rallies + the
136         dead # time between them. Never merges, only cuts (recall can't drop). Measured
137         recommended value # 12.0 (golden n=6: ?F1 +0.139, 6/0 sign-test, merge_rate 0.523?0.038); kept OFF
138         until promoted. max_window_duration: float = 0.0
139         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
140         point # HARD-CAP fallback for W1: when True, an over-long window with NO internal
141         inactivity gap # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
142         is # chopped at its midpoint until ? max_window_duration, making the cap a true upper
143         bound. # Only cuts (recall can't drop). OFF by default until measured/promoted.
144         window_hard_cap: bool = False
145         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
146         slowly # (picked up / walked) above velocity_thresh, so the window over-extends its END
147         (the measured # [?end| gap vs golden). Trim the post-rally tail back to the last frame whose
148         trailing # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
149         cuts. OFF by # default. Measured win at inactivity_end=1.5/rally_thresh=0.20/min_density=0.30
150         (n=11: ?tolF1 # +0.047, sign 8/0 p=0.008, [?end| 5.7?3.8s) ? see docs/RALLY_QUALITY_RESEARCH.md.
151         window_inactivity_end: float = 0.0
152         inactivity_rally_thresh: float = 0.08
153         inactivity_min_density: float = 0.34
154         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
155         blob: after # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
156         STILL longer # than window_blob_factor × max_window_duration (no internal gap, no rally-end
157         signal ? e.g. # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
158         a FORCED cut # (logged + flagged on the window) so the degenerate single-window outcome is
159         avoided and the # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
160         window_hard_cap

```

```

160         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
161         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
162         window_blob_fallback: bool = False
163         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
164         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
165         # Default 4.0 is do-no-harm on the n=11 golden corpus (rescues only the 2
continuously-active
166         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
167         window_blob_factor: float = 4.0
168         log_yolo_candidates: bool = True
169         store_yolo_candidates: bool = True
170         ai_handoff_padding: float = 2.0
171         ai_max_workers: int = 5
172         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
173         # chunks of a high-bitrate (4K) source blow past the provider upload cap
174         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
175         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
176         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
177         # Matches gemini_refine's --clip-scale-width default.
178         ai_chunk_scale_width: int = 1280
179         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
180         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
181         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
182         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
183         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
184         skip_ai_handoff: bool = False
185         # Optional debug knob: trim every video to the first N seconds before processing.
186         debug_duration: Optional[float] = None
187
188         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
189         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
190         fusion: FusionConfig = Field(default_factory=FusionConfig)
191
192         @field_validator("default_segmenter")
193         @classmethod
194         def segmenter_must_be_registered(cls, v: str) -> str:
195             # Registry check happens at runtime in main/segmenter selection.
196             # Here we just allow any string for forward compatibility.
197             return v
198
199         @model_validator(mode="after")
200         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
201             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
202             # step makes `while t < duration: t += step` loop forever, growing memory before
any
203             # GPU work. Reject the bad config at load time instead.
204             if self.chunk_overlap >= self.chunk_duration:
205                 raise ValueError(
206                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
207                     f"({self.chunk_duration}); otherwise chunking never advances."
208                 )
209             return self
210
211
212     class OutputConfig(BaseModel):

```

```

213     default_highlights_name: str = "highlights.mp4"
214     db_name: str = "sports_indexer.db"
215     # Directory where the DB, highlight reels, and processed clips are written.
216     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
217     output_dir: str = "output"
218
219
220     class WatermarkConfig(BaseModel):
221         """Branding overlay burned into each highlight clip during the per-clip re-encode
222         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
223         (under `stitching.watermark` in config.json) to turn it off. The text is fully
224         configurable. The concat stage stays stream-copy (-c copy)."""
225         enabled: bool = True
226         text: str = "Generated by Khelsutra.guru"
227         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
228         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
229         font: str = "Sans" # fontconfig family used when font_path is empty
230         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
231         # fraction of frame height
232         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
233         box: bool = True # faint backing box behind the text for legibility
234         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
235         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
236         right"
237
238     class StitchingConfig(BaseModel):
239         begin_padding: float = 0.5
240         end_padding: float = 0.5
241         use_cache: bool = False
242         min_shot_count: int = 1
243         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
244
245     class LoggingConfig(BaseModel):
246         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
247         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
248         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
249         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
250         # them to WARNING; set true to restore full chatter for request-flow debugging.
251         verbose_http: bool = False
252
253
254     class SecurityConfig(BaseModel):
255         # When set, /api/process video_path must resolve under this directory (hosted/tenant
256         # mode). Default None preserves the single-user local 'any local file' workflow.
257         allowed_video_root: Optional[str] = None
258
259         # --- Chunked upload (B2, PR-2) ---
260         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
261         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
262         # contain upload_dir or /api/process will reject the uploaded paths.
263         upload_dir: str = "output/uploads"
264         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
265         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
266         max_upload_mb: int = 4096
267         max_part_mb: int = 95
268         allowed_upload_extensions: List[str] = ["mp4", "mov"]
269
270
271     class AppConfig(BaseModel):
272         """Root configuration object.
273
274         All runtime configuration should be accessed via an instance of this class

```



```

275         (or the loader) rather than raw dict lookups.
276         """
277
278         sport: Sport = "badminton"
279         ai_provider: str = "gemini"
280         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
281         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
282         ai_model: str = "gemini-flash-latest"
283
284         validation: ValidationConfig = Field(default_factory=ValidationConfig)
285         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
286         output: OutputConfig = Field(default_factory=OutputConfig)
287         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
288         security: SecurityConfig = Field(default_factory=SecurityConfig)
289         logging: LoggingConfig = Field(default_factory=LoggingConfig)
290
291         # Allow extra keys during transition so existing config.json files do not break.
292         model_config = {
293             "extra": "allow",
294             "validate_assignment": True,
295         }
296
297         # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
298         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
299         # attribute access / the typed config. A nested *section* is returned as a plain
dict
300         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
301         # section model instead breaks it (a model has no .get), which bit every validator
on
302         # real runs. Scalar leaves are returned as-is.
303         def get(self, key: str, default: Any = None) -> Any:
304             _MISSING = object()
305             val: Any = _MISSING
306             if key in type(self).model_fields or hasattr(self, key):
307                 val = getattr(self, key, _MISSING)
308             if val is _MISSING:
309                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
310                 # discovered dynamically from the model fields, so a future top-level
section
311                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
312                 for fname in type(self).model_fields:
313                     section = getattr(self, fname, None)
314                     if isinstance(section, BaseModel) and hasattr(section, key):
315                         val = getattr(section, key)
316                         break
317             if val is _MISSING:
318                 return default
319             if isinstance(val, BaseModel):
320                 return val.model_dump(mode="json")
321             return val
322
323         def __getitem__(self, key: str) -> Any:
324             val = self.get(key)
325             if val is None and key not in type(self).model_fields:
326                 # Re-raise like a real dict for unknown top-level
327                 raise KeyError(key)
328             return val
329
330
331         # Convenience type alias for callers that want to annotate

```

```
332     Config = AppConfig
333
```

21. assistant (2026-06-17T14:35:47.985Z)

Perfect! Now let me check the test file to see if there are tests verifying the watermark behavior:

22. user (2026-06-17T14:35:49.607Z)

```
./tests/test_generate_highlights.py
```

23. user (2026-06-17T14:35:51.042Z)

```
1     """tools/generate_highlights.py ? the batch highlight runner. The load-bearing
guarantees
2     are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
3     (heavy deps ? segmenter/compiler/db ? are mocked)."""
4     import os
5     from unittest.mock import MagicMock, patch
6
7     import tools.generate_highlights as gh
8
9
10    def test_atomic_publish_is_nondestructive(tmp_path):
11        src = tmp_path / "local_reel.mp4"
12        src.write_bytes(b"reel-bytes")
13        dest_dir = tmp_path / "out"
14        dest_dir.mkdir()
15        sibling = dest_dir / "Someone Else - Highlights.mp4"
16        sibling.write_bytes(b"do-not-touch")
17
18        gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))
19
20        assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
21        assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
22        assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind
23
24
25    def test_resolve_videos_explicit_and_dir(tmp_path):
26        (tmp_path / "a.MP4").write_bytes(b"x")
27        (tmp_path / "b.mp4").write_bytes(b"x")
28        (tmp_path / "notes.txt").write_text("ignore")
29        args = MagicMock(video=None, videos_dir=str(tmp_path))
30        found = gh._resolve_videos(args)
31        assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]  # videos only,
sorted
32        args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
33        assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]
34
35
36    def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
37        """force=False: a video whose reel already exists is skipped, the segmenter is never
38        called, and unrelated files already in the destination are left intact."""
39        out = tmp_path / "out"
40        out.mkdir()
41        (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
42        (out / "Adarsh v Avi - run.log").write_text("prior log")
43        other = out / "Someone Else - Highlights.mp4"
44        other.write_bytes(b"untouched")
45        src = tmp_path / "Adarsh v Avi.MP4"
46        src.write_bytes(b"src")
47
48        seg_registry = MagicMock()
```

```

49         with patch.object(gh, "load_config", return_value={}), \
50             patch.object(gh, "Database", return_value=MagicMock()), \
51             patch.object(gh, "segmenter_registry", seg_registry):
52             summary = gh.generate_highlights_batch(
53                 [str(src)], str(out), segmenter="gemini",
54                 local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
55                 force=False)
56             assert summary["skipped"] == ["Adarsh v Avi.MP4"]
57             # never processed: the resolved segmenter class was never instantiated/run
58             seg_registry.get.return_value.assert_not_called()
59             assert other.read_bytes() == b"untouched" # other work preserved
60             assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"
61             # not regenerated
62
63     def test_batch_applies_local_overrides(tmp_path):
64         """--skip-ai-handoff / --wasb-stream inject into the typed config before
65         segmentation."""
66         from backend.config import AppConfig
67         cfg = AppConfig()
68         assert cfg.indexing.skip_ai_handoff is False # default off
69         assert cfg.indexing.wasb.stream_video is False
70
71         out = tmp_path / "out"
72         out.mkdir()
73         src = tmp_path / "clip.MP4"
74         src.write_bytes(b"src")
75
76         seg_instance = MagicMock()
77         seg_instance.process_video.return_value = [] # no rallies -> no
78         compile/publish
79         seg_registry = MagicMock()
80         seg_registry.get.return_value = MagicMock(return_value=seg_instance)
81
82         with patch.object(gh, "load_config", return_value=cfg), \
83             patch.object(gh, "Database", return_value=MagicMock()), \
84             patch.object(gh, "compute_video_id", return_value="vid123"), \
85             patch.object(gh, "segmenter_registry", seg_registry):
86             gh.generate_highlights_batch(
87                 [str(src)], str(out), segmenter="gemini",
88                 local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
89                 force=True, skip_ai_handoff=True, wasb_stream=True)
90
91         assert cfg.indexing.skip_ai_handoff is True
92         assert cfg.indexing.wasb.stream_video is True
93

```

24. assistant (2026-06-17T14:35:52.691Z)

Now let me check what the default watermark config is and if there are any tests related to the watermark in the commit:

25. user (2026-06-17T14:35:53.477Z)

```

commit a6adcc57290bd65c3ca7c092d0568ff02cf60378
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Wed Jun 17 19:59:43 2026 +0530

```

fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels

The batch runner (tools/generate_highlights.py) constructed VideoCompiler bare ?
`VideoCompiler(local\_work\_dir).compile\_highlights(...)` ? so reels shipped UN-watermarked
(watermark defaults to None/off) and used the ctor's begin/end padding (0.5/1.0) instead of
config.json `stitching.{begin\_padding,end\_padding}` (0.5/0.5). Every other compile call site

threads stitching config; this one didn't. Now it reads `stitching.\*` dict-compatibly and passes watermark + paddings through (the on-by-default branding mark).

Also add `--detect-from`: run the detector on a cheaper proxy (e.g. a 1080p downscale of a 4K source) while stitching the reel from the full-res --video. Timestamps transfer 1:1 (pure temporal-preserving downscale), so it's the bandwidth/GPU-saver split the workflow wanted ? detect on the proxy, deliver a 4K reel. Reel is named after the --video (the thing watched).

Tests: extend test_generate_highlights.py ? watermark+padding are threaded into VideoCompiler (regression lock), --detect-from keys detection off the proxy but stitches from the original, and _resolve_detect_from validates 1:1 pairing.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

```
diff --git a/tests/test_generate_highlights.py b/tests/test_generate_highlights.py
index b0f5d35..ed92f39 100644
--- a/tests/test_generate_highlights.py
+++ b/tests/test_generate_highlights.py
@@ -4,9 +4,30 @@ are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock
h
import os
from unittest.mock import MagicMock, patch

+import pytest
+
+import tools.generate_highlights as gh

+def _fake_compiler(captured):
+    """A stand-in VideoCompiler that records its ctor kwargs + compile args and writes the
+    expected reel file (so the runner's atomic-publish step succeeds)."""
+    class FakeCompiler:
+        def __init__(self, output_dir, end_padding=1.0, begin_padding=0.5, watermark=None):
+            captured["ctor"] = dict(output_dir=output_dir, end_padding=end_padding,
+                                   begin_padding=begin_padding, watermark=watermark)
+
+        def compile_highlights(self, raw_video_path, segments, output_filename):
+            captured["compile"] = dict(raw=raw_video_path, segments=list(segments),
+                                      out=output_filename)
+            reel = os.path.join(captured["ctor"]["output_dir"], output_filename)
+            with open(reel, "wb") as fh:
+                fh.write(b"reel-bytes")
+            return reel
+
+    return FakeCompiler

def test_atomic_publish_is_nondestructive(tmp_path):
    src = tmp_path / "local_reel.mp4"
    src.write_bytes(b"reel-bytes")
@@ -88,3 +109,71 @@ def test_batch_applies_local_overrides(tmp_path):

    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.indexing.wasb.stream_video is True
+
+
+def _run_one(tmp_path, *, video_paths, detect_from_paths=None, segmenter="gemini"):
+    """Drive generate_highlights_batch once with a fake compiler/segmenter/db, returning the
+    captured VideoCompiler ctor+compile args and the compute_video_id mock."""
+    from backend.config import AppConfig
+    cfg = AppConfig() # watermark on by default
+    out = tmp_path / "out"
```

```

+ out.mkdir()
+ db = MagicMock()
+ db.query_play_segments.return_value = [{"start_time": 1.0, "end_time": 5.0}]
+ seg_instance = MagicMock()
+ seg_instance.process_video.return_value = ["ok"] # non-Failure
+ seg_registry = MagicMock()
+ seg_registry.get.return_value = MagicMock(return_value=seg_instance)
+ captured: dict = {}
+ cvid = MagicMock(return_value="vid123")
+ with patch.object(gh, "load_config", return_value=cfg), \
+     patch.object(gh, "Database", return_value=db), \
+     patch.object(gh, "compute_video_id", cvid), \
+     patch.object(gh, "VideoCompiler", _fake_compiler(captured)), \
+     patch.object(gh, "segmenter_registry", seg_registry):
+     gh.generate_highlights_batch(
+         [str(p) for p in video_paths], str(out), segmenter=segmenter,
+         local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
+         force=True, detect_from_paths=[str(p) for p in detect_from_paths] if
detect_from_paths else None)
+     return captured, cvid, out
+
+
+def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
+     """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to
VideoCompiler
+     (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
+     src = tmp_path / "clip.MP4"
+     src.write_bytes(b"src")
+     captured, _cvid, out = _run_one(tmp_path, video_paths=[src])
+
+     wm = captured["ctor"]["watermark"]
+     assert wm and wm.get("enabled") is True # on-by-default mark threaded
through
+     assert wm.get("text") == "Generated by Khelsutra.guru"
+     assert captured["ctor"]["begin_padding"] == 0.5 # config paddings, not the ctor 1.0
default
+     assert captured["ctor"]["end_padding"] == 0.5
+     assert captured["compile"]["raw"] == str(src) # stitches from the (only) source
+     assert (out / "clip - Highlights.mp4").exists() # published
+
+
+def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
+     """--detect-from: the detector runs on the proxy (video_id keyed off it) but the reel is
+     stitched from the full-res --video, and named after it."""
+     orig = tmp_path / "GX01.MP4"
+     orig.write_bytes(b"original-4k")
+     proxy = tmp_path / "GX01_1080p.mp4"
+     proxy.write_bytes(b"proxy-1080")
+     captured, cvid, out = _run_one(tmp_path, video_paths=[orig], detect_from_paths=[proxy])
+
+     cvid.assert_called_once_with(str(proxy)) # detection keyed off the proxy
+     assert captured["compile"]["raw"] == str(orig) # reel stitched from the original
+     assert (out / "GX01 - Highlights.mp4").exists() # named after the original, not the
proxy
+
+
+def test_resolve_detect_from_validates_pairing(tmp_path):
+     assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
+     with pytest.raises(SystemExit): # count mismatch
+         gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
+     with pytest.raises(SystemExit): # requires explicit --video
+         gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None, videos_dir="d"),
["v1"])
+     abs = os.path.join(os.sep, "abs", "p.mp4")

```

```
+ assert gh._resolve_detect_from(
+     MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
```

26. assistant (2026-06-17T14:35:57.566Z)

Excellent! Now I have everything I need. Let me verify the key facts:

1. **Old code (before commit):** ``VideoCompiler(local_work_dir).compile_highlights(...)``
2. **New code (after commit):**

```
```python
st = cfg.get("stitching") or {}
VideoCompiler(
 local_work_dir,
 begin_padding=float(st.get("begin_padding", 0.5)),
 end_padding=float(st.get("end_padding", 0.5)),
 watermark=st.get("watermark"),
).compile_highlights(stitch_src, pairs, reel_name)
```
```
3. **VideoCompiler signature** (line 17-18 of compiler.py):

```
```python
def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
 watermark: Optional[Any] = None):
...`
```
4. **Default WatermarkConfig** (from models.py line 225):

```
```python
class WatermarkConfig(BaseModel):
    enabled: bool = True
    text: str = "Generated by Khelsutra.guru"
...`
```
5. **StitchingConfig default** (line 242 of models.py):

```
```python
watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
...`
```

The finding claims that ``st.get("watermark")`` returns the full watermark dict including `{enabled:true, text:...}``. Let me verify this is actually what happens:

## 27. user (2026-06-17T14:35:58.547Z)

```
tmp = dest_path + ".publishing.tmp"
shutil.copy2(local_path, tmp)
os.replace(tmp, dest_path)
```

```
def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
 local_work_dir: str, db_path: str, force: bool = False,
 skip_ai_handoff: bool = False, wasb_stream: bool = False,
 detect_from_paths: Optional[List[str]] = None) -> dict:
 """Generate one highlight reel per input video into out_dir. Returns a summary dict.
```

Non-destructive: only ADDS reels/logs to out\_dir (atomic publish); never deletes.

`skip_ai_handoff`: run the trajectory-hybrid segmenters fully-local (no Gemini handoff); candidate windows become the rallies. `wasb_stream`: have the WASB detector decode the video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178). `detect_from_paths`: optional list parallel to video\_paths. When given, the detector runs on `detect_from_paths[i]` (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the reel is stitched from `video_paths[i]` at full resolution. Rally timestamps transfer 1:1 (a pure temporal-preserving downscale shares the timeline). `None` = detect+stitch the same source (legacy behaviour).

## 28. assistant (2026-06-17T14:36:00.121Z)

Now let me look at the actual implementation around where watermark is used:

## 29. user (2026-06-17T14:36:00.675Z)

```
NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
ctor default paddings; this threads config.json through (the on-by-default mark).
st = cfg.get("stitching") or {}
VideoCompiler(
 local_work_dir,
 begin_padding=float(st.get("begin_padding", 0.5)),
 end_padding=float(st.get("end_padding", 0.5)),
 watermark=st.get("watermark"),
).compile_highlights(stitch_src, pairs, reel_name)
_atomic_publish(local_reel, dest_reel)
try:
 os.remove(local_reel)
except OSError:
 pass
logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
summary["done"].append(name)
except Exception:
 logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
 summary["failed"].append(name)
finally:
 logging.getLogger().removeHandler(run_fh)
 run_fh.close()
publish this video's run.log (additive; never deletes anything in out_dir)
```

## 30. assistant (2026-06-17T14:36:08.536Z)

Perfect! Now I have a complete picture. Let me trace through what happens:

1. `cfg = load\_config()` loads the typed AppConfig
2. `st = cfg.get("stitching") or {}` - let's check what `cfg.get("stitching")` returns

Looking at the `AppConfig.get()` method in models.py (line 303-321):

```
```python
def get(self, key: str, default: Any = None) -> Any:
    # ...
    val = getattr(self, key, _MISSING)
    # ...
    if isinstance(val, BaseModel):
        return val.model_dump(mode="json") # <-- This line is key!
    return val
```
```

So when you call `cfg.get("stitching")`, it returns `val.model_dump(mode="json")` which is a dict representation of the `StitchingConfig` pydantic model.

This means:

- `st` is a dict
- `st.get("watermark")` returns a dict (the `model_dump` of `WatermarkConfig`), which by default includes `{"enabled": True, "text": "Generated by Khelsutra.guru", ...}`
- This dict is passed to `VideoCompiler(..., watermark=st.get("watermark"))`

Now looking at `VideoCompiler's` `__init__` (compiler.py line 17-24):

```
```python
def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
             watermark: Optional[Any] = None):
    # ...
    self.watermark = self._normalize_watermark(watermark)
```
```

@staticmethod

```
def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
 if watermark is None:
 return None
 if hasattr(watermark, "model_dump"):
 return watermark.model_dump()
 if isinstance(watermark, dict):
 return dict(watermark)
 return None
...
```

So when `watermark` is a dict (which it is from `st.get("watermark")`), `\_normalize\_watermark` returns `dict(watermark)` - the dict copy.

**\*\*The key issue:\*\*** Before this change, watermark was None (default parameter). After, it's the full watermark dict. The test confirms this is intentional:

```
```python
def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
    """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to VideoCompiler
    (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
    # ...
    wm = captured["ctor"]["watermark"]
    assert wm and wm.get("enabled") is True                # on-by-default mark threaded
through
```
```

This is **\*\*REAL and INTENTIONAL\*\*** - it's a fix to enable watermarking that was previously disabled. The commit message explicitly states this is the goal: "threads config.json through (the on-by-default mark)". The tests explicitly verify that watermark is now passed through.

This is NOT a bug - it's the intended behavior change documented in the commit message and tested.

**31. user (2026-06-17T14:36:12.064Z)**

Structured output provided successfully